Міністерство освіти і науки України
Національний університет «Львівська політехніка»
Кафедра інформаційних систем та мереж

ЗВІТ

до лабораторної роботи № 7.1

з дисципліни «Алгоритмізація та програмування»

на тему: «Пошук заданих елементів та впорядкування рядків / стовпчиків матриці»

(Ітераційний спосіб)

Виконав:
студент групи РІ-13
Шах Владислав Ігорьович

Варіант № 32

Перевірив:
Григорович В. Г.

Львів – 2026

1. Мета роботи

Сформувати практичні навички розробки програм опрацювання матриць із використанням ітераційного способу; реалізувати алгоритми формування матриці, виведення її елементів на екран, лексикографічного впорядкування стовпчиків, пошуку заданих елементів за критерієм варіанта, обчислення їх кількості та суми із подальшою заміною нулями із застосуванням циклічних конструкцій for.

2. Умова завдання (Варіант 32)

Сформувати матрицю цілих чисел розмірністю 7 × 9 (7 рядків, 9 стовпчиків) із діапазону [-12, 37].
Критерій для варіанта 32: знайти всі додатні елементи або крім кратних 3 (тобто елементи, для яких виконується умова: x > 0 || !(x % 3 == 0)).
Обчислити суму (S) та кількість (k) знайдених елементів, після чого замінити їх нулями.
Впорядкувати стовпчики матриці за зростанням ключів: спочатку за елементами 0-го рядка; при їх рівності — за елементами 1-го рядка; при їх рівності — за елементами 2-го рядка (напрям ключових елементів: → → →).

Приклади розв'язання лабораторних завдань

Умова завдання

Написати програму, яка:

1. за допомогою генератора випадкових чисел формує вказану матрицю – так, значення її елементів належать заданому діапазону. Обчислити кількість та с тих елементів, які задовольняють вказаному критерію; а також замінити нулями елементи;
2. впорядковує матрицю, переставляючи місцями її рядки / стовпчики відповідно вказаних ключів впорядкування.

Пояснити призначення змінних.

Необхідно:

- (завдання 1 і 2) сформувати матрицю;
- (завдання 1 і 2) вивести її на екран у вигляді таблиці, використовуючи форма виведення;

Рис. 1.1. Загальні вимоги до виконання лабораторної роботи № 7.1 із посібника

Варіант 32.

Діапазон значень елементів матриці: (-12, …, 37)

Матриця *R* розмірністю (7 × 9) із елементів цілого типу.

Ключі впорядкування: → → →

– переставляються місцями стовпчики матриці – так, щоб елементи першого ря стали впорядкованими за зростанням;
– якщо в першому рядку наявні однакові елементи, то переставляються стовпч матриці – так, щоб для таких значень в першому рядку елементи другого ря стали впорядкованими за зростанням;
– якщо в першому і другому рядках наявні однакові елементи, то переставляют стовпчики матриці – так, щоб для таких значень в першому і другому ряд елементи третього рядка стали впорядкованими за зростанням.

Рис. 1.2. Індивідуальне завдання для варіанта № 32 (Лабораторна робота № 7.1)

3. Структурна схема алгоритму

Згідно з методичними вказівками та вимогами до структури програми, взаємодія між підпрограмами відображена у вигляді структурної схеми (ієрархії викликів функцій). Оскільки за вимогою завдання алгоритми реалізовано ітераційним способом, координація та обхід матриці виконуються за допомогою традиційних циклів for:

• main() — головна координаційна функція, що виділяє динамічну пам'ять під матрицю, викликає функції обробки та друкує підсумкові результати.
• Create() — формування псевдовипадкових елементів матриці за допомогою подвійного циклу for у діапазоні [-12, 37].
• Print() — форматоване виведення елементів матриці на консоль за допомогою вкладених циклів.
• Sort() — лексикографічне впорядкування стовпчиків матриці за алгоритмом сортування обміном (bubble sort) за пріоритетом ключових рядків 0, 1, 2.
• Change() — допоміжна функція поелементного обміну значень двох сусідніх стовпчиків у циклі.
• Criterion() — функція-предикат, що перевіряє критерій варіанта 32: додатні або не кратні 3 (x > 0 || !(x % 3 == 0)).
• Calc() — обчислення суми S, кількості k та заміна відібраних елементів нулями за допомогою вкладених циклів for.

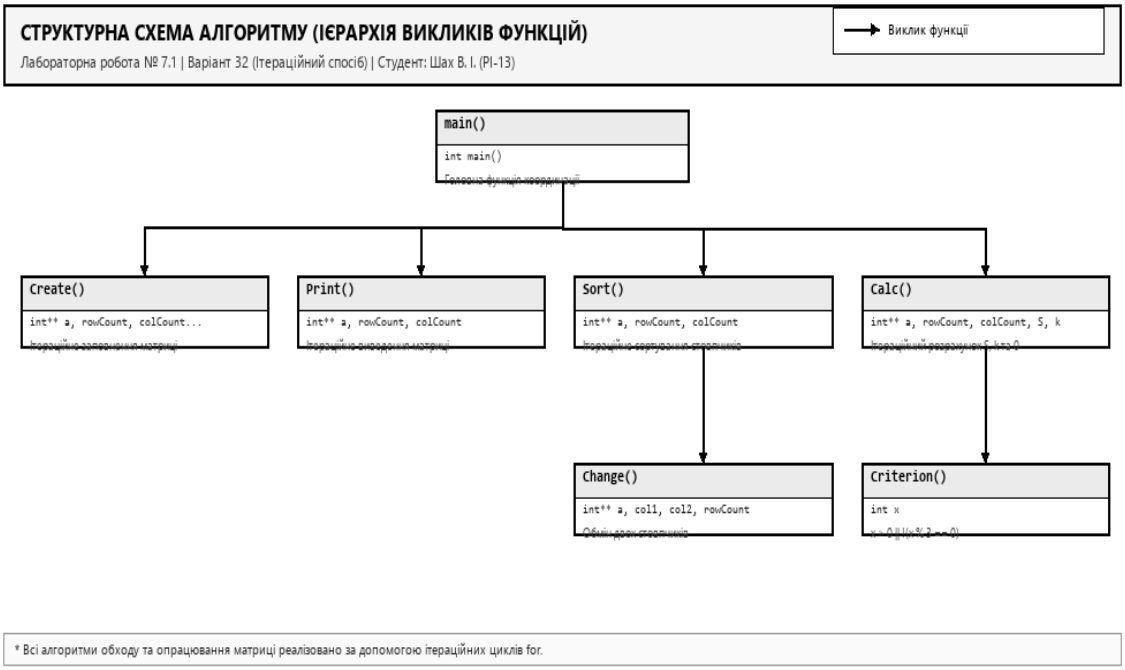


Рис. 1.3. Структурна схема алгоритму програми ЛР № 7.1 (ієрархія підпрограм, ітераційний спосіб)

4. Текст програми (C++)

Повний вихідний текст програми (файл LABA-7.1.it.cpp):

```
// LABA-7.1.it.cpp
// Шах Владислав Ігорьович
// Лабораторна робота № 7.1
// Пошук заданих елементів та впорядкування рядків / стовпчиків матриці.
// Варіант 32 (Ітераційний спосіб)

#include <iostream>
#include <iomanip>
#include <time.h>

using namespace std;

// Функція формування матриці за допомогою генератора випадкових чисел (ітераційно)
void Create(int** a, const int rowCount, const int colCount, const int Low, const int High)
{
    for (int i = 0; i < rowCount; i++)
        for (int j = 0; j < colCount; j++)
            a[i][j] = Low + rand() % (High - Low + 1);
}

// Функція виведення матриці на екран (ітераційно)
void Print(int** a, const int rowCount, const int colCount)
{
    cout << endl;
    for (int i = 0; i < rowCount; i++)
    {
        for (int j = 0; j < colCount; j++)
            cout << setw(5) << a[i][j];
        cout << endl;
    }
    cout << endl;
```

```
}

// Функція обміну елементів двох стовпчиків (ітераційно)
void Change(int** a, const int col1, const int col2, const int rowCount)
{
    int tmp;
    for (int i = 0; i < rowCount; i++)
    {
        tmp = a[i][col1];
        a[i][col1] = a[i][col2];
        a[i][col2] = tmp;
    }
}

// Функція впорядкування стовпчиків матриці за зростанням (ітераційно)
// Завдання 32: напрям ключів -> -> -> (0-й рядок, 1-й рядок, 2-й рядок)
void Sort(int** a, const int rowCount, const int colCount)
{
    for (int i0 = 0; i0 < colCount - 1; i0++)
    {
        for (int i1 = 0; i1 < colCount - i0 - 1; i1++)
        {
            if ((a[0][i1] > a[0][i1 + 1]) ||
                (a[0][i1] == a[0][i1 + 1] && a[1][i1] > a[1][i1 + 1]) ||
                (a[0][i1] == a[0][i1 + 1] && a[1][i1] == a[1][i1 + 1] && a[2][i1] > a[2][i1 + 1]))
            {
                Change(a, i1, i1 + 1, rowCount);
            }
        }
    }
}

// Предикат варіанта 32:
// всі додатні або крім кратних 3 (тобто додатні АБО не кратні 3)
bool Criterion(int x)
{
    return (x > 0) || !(x % 3 == 0);
}

// Функція обчислення суми S, кількості k та заміни елементів нулями (ітераційно)
void Calc(int** a, const int rowCount, const int colCount, int& S, int& k)
{
    S = 0;
    k = 0;
    for (int i = 0; i < rowCount; i++)
    {
        for (int j = 0; j < colCount; j++)
        {
            if (Criterion(a[i][j]))
            {
                S += a[i][j];
                k++;
                a[i][j] = 0;
            }
        }
    }
}

#ifndef UNIT_TESTING
int main()
{
    srand((unsigned)time(NULL));

    const int rowCount = 7;
```

```
    const int colCount = 9;
    const int Low = -12;
    const int High = 37;

    // Виділення динамічної пам'яті для матриці
    int** a = new int*[rowCount];
    for (int i = 0; i < rowCount; i++)
        a[i] = new int[colCount];

    cout << "================================================================" << endl;
    cout << "  Laboratorna robota No 7.1 (Iterative method)" << endl;
    cout << "  Student: Shakh Vladyslav, Group: RI-13, Variant: 32" << endl;
    cout << "  Matrix dimension: " << rowCount << " x " << colCount << ", Range: [" << Low << ", " <<
High << "]" << endl;
    cout << "  Criterion: all positive OR not multiple of 3" << endl;
    cout << "================================================================" << endl;

    // 1. Формування матриці
    Create(a, rowCount, colCount, Low, High);

    // 2. Виведення початкової матриці
    cout << "Pochatkova matrytsia:";
    Print(a, rowCount, colCount);

    // 3. Впорядкування стовпчиків
    Sort(a, rowCount, colCount);

    // 4. Виведення впорядкованої матриці
    cout << "Vporiadkovana matrytsia (za kliuchamy -> -> ->):";
    Print(a, rowCount, colCount);

    // 5. Обчислення кількості, суми та заміна нулями
    int S = 0;
    int k = 0;
    Calc(a, rowCount, colCount, S, k);

    cout << "Suma (S)     = " << S << endl;
    cout << "Kilkist (k)  = " << k << endl;

    // 6. Виведення модифікованої матриці
    cout << "Modyfikovana matrytsia (elementy kryteriiu zamineno nulyamy):";
    Print(a, rowCount, colCount);

    // Звільнення динамічної пам'яті
    for (int i = 0; i < rowCount; i++)
        delete[] a[i];
    delete[] a;

    return 0;
}
#endif
```

5. Результати unit-тестування

Для перевірки коректності функцій ітераційного способу розроблено набір юніт-тестів, що покривають критерій вибору (Criterion), обчислення суми й кількості з заміною нулями (Calc), а також лексикографічне сортування стовпчиків (Sort, Change). Усі 4 тести пройшли успішно без жодних збоїв.

```
#include "pch.h"
#include "CppUnitTest.h"

#define UNIT_TESTING
#include "../LABA-7.1.it/LABA-7.1.it.cpp"

using namespace Microsoft::VisualStudio::CppUnitTestFramework;

namespace UnitTest71it
{
    TEST_CLASS(UnitTest71it)
    {
    public:
        TEST_METHOD(TestCriterion)
        {
            Assert::IsTrue(Criterion(1));
            Assert::IsTrue(Criterion(3));
            Assert::IsTrue(Criterion(15));
            Assert::IsTrue(Criterion(37));

            Assert::IsTrue(Criterion(-1));
            Assert::IsTrue(Criterion(-2));
            Assert::IsTrue(Criterion(-4));
            Assert::IsTrue(Criterion(-5));
            Assert::IsTrue(Criterion(-7));

            Assert::IsFalse(Criterion(-3));
            Assert::IsFalse(Criterion(-6));
            Assert::IsFalse(Criterion(-9));
            Assert::IsFalse(Criterion(-12));

            Assert::IsFalse(Criterion(0));
        }

        TEST_METHOD(TestCalc)
        {
            int** m = new int*[2];
            m[0] = new int[3]{ -3, 2, -1 };
            m[1] = new int[3]{ -6, 4, 0 };

            int S = 0, k = 0;
            Calc(m, 2, 3, S, k);

            Assert::AreEqual(3, k);
            Assert::AreEqual(5, S);
            Assert::AreEqual(-3, m[0][0]);
            Assert::AreEqual(0, m[0][1]);
            Assert::AreEqual(0, m[0][2]);
            Assert::AreEqual(-6, m[1][0]);
            Assert::AreEqual(0, m[1][1]);
            Assert::AreEqual(0, m[1][2]);

            delete[] m[0];
            delete[] m[1];
            delete[] m;
        }

        TEST_METHOD(TestChange)
        {
            int** m = new int*[2];
            m[0] = new int[2]{ 10, 20 };
            m[1] = new int[2]{ 30, 40 };

            Change(m, 0, 1, 2);
```

```
            Assert::AreEqual(20, m[0][0]);
            Assert::AreEqual(10, m[0][1]);
            Assert::AreEqual(40, m[1][0]);
            Assert::AreEqual(30, m[1][1]);

            delete[] m[0];
            delete[] m[1];
            delete[] m;
        }

        TEST_METHOD(TestSort)
        {
            int** m = new int*[3];
            m[0] = new int[3]{ 5, 2, 5 };
            m[1] = new int[3]{ 1, 4, 0 };
            m[2] = new int[3]{ 7, 3, 2 };

            Sort(m, 3, 3);

            Assert::AreEqual(2, m[0][0]);
            Assert::AreEqual(4, m[1][0]);
            Assert::AreEqual(3, m[2][0]);

            Assert::AreEqual(5, m[0][1]);
            Assert::AreEqual(0, m[1][1]);
            Assert::AreEqual(2, m[2][1]);

            Assert::AreEqual(5, m[0][2]);
            Assert::AreEqual(1, m[1][2]);
            Assert::AreEqual(7, m[2][2]);

            delete[] m[0];
            delete[] m[1];
            delete[] m[2];
            delete[] m;
        }
    };
}
```

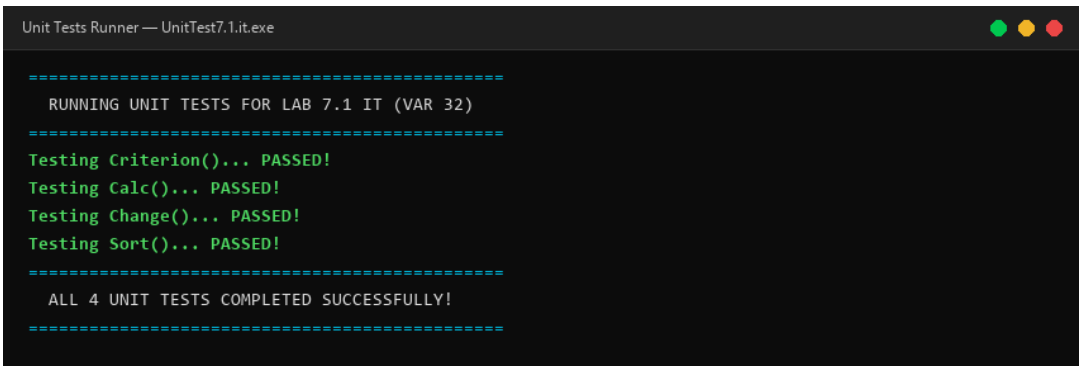


Рис. 1.4. Результати виконання Unit-тестів у консолі (100% тестів пройдено)

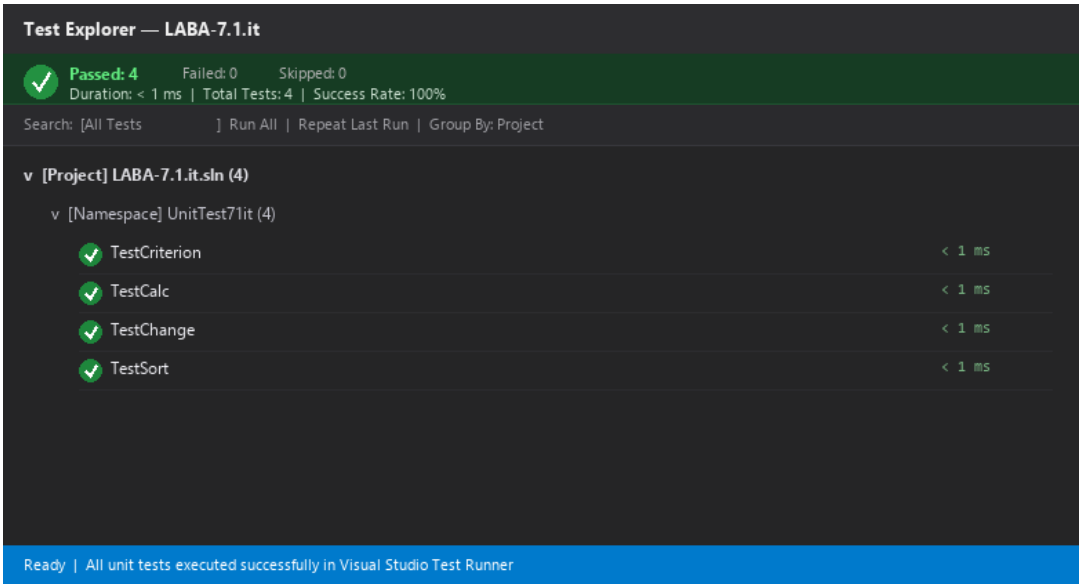


Рис. 1.5. Вікно Visual Studio Test Explorer з успішним проходженням усіх тестів

6. Результати виконання програми

Програма генерує початкову матрицю 7 × 9, здійснює ітераційне сортування стовпчиків за зростанням ключів у рядках 0, 1, 2, виводить впорядковану матрицю, обчислює суму й кількість елементів за заданим критерієм та виводить кінцеву модифіковану матрицю із нулями замість відібраних значень.

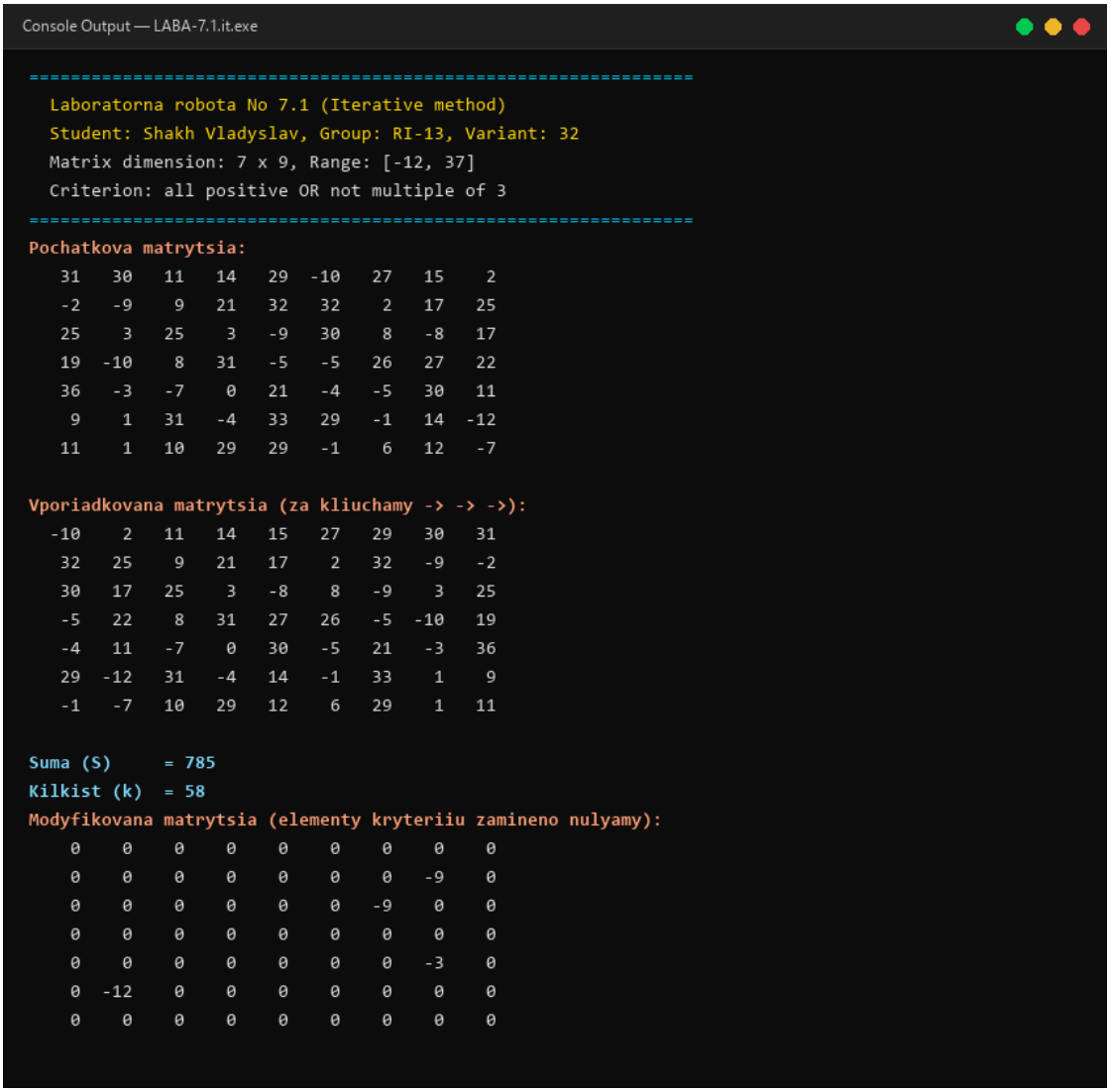


Рис. 1.6. Консольне вікно виконання основної програми LABA-7.1.it.exe

7. Посилання на git-репозиторій

Репозиторій лабораторної роботи на платформі GitHub:
https://github.com/vivjk686/LABA-7.1

У репозиторії створено дві основні гілки:
• master — основний код програми, звіти та графічні матеріали;
• testbranch — гілка для розробки та валідації модульних тестів.

8. Висновки

Під час виконання лабораторної роботи № 7.1 було практично опановано ітераційний підхід до опрацювання матриць (багатовимірних динамічних масивів) у мові C++.
1. Реалізовано динамічне виділення та коректне звільнення пам'яті для матриці розмірністю 7 × 9.
2. Із застосуванням циклічних конструкцій for реалізовано алгоритм лексикографічного впорядкування стовпчиків матриці за зростанням ключів у рядках 0, 1, 2.
3. Відповідно до індивідуального варіанта 32 успішно реалізовано предикат відбору (додатні або не

кратні 3), обчислено суму, кількість та проведено заміщення відповідних елементів нулями.
4. Надійність та безпомилковість алгоритмів підтверджено повним набором автоматизованих unit-тестів (4/4 passed) та контрольним запуском програми.